

Olá! Seja bem-vindo e bem-vinda à disciplina de Programação de Computadores e aos nossos estudos em estrutura de dados básicas em Python. Até aqui você aprendeu a trabalhar com variáveis, tipos de dados, simples e estruturas de controle. Agora daremos um passo muito importante: organizar vários dados ao mesmo tempo, de forma eficiente e profissional. Pense em uma lista de compras, uma agenda de contatos ou um sistema de cadastro. Agora imagine isso em sistemas complexos. Afinal, programas modernos lidam com grandes volumes de informação. Sem estruturas de dados, cada valor teria que ser armazenado em uma variável diferente, tornando o código confuso e inviável. As estruturas de dados existem para organizar, acessar e manipular informações de forma eficiente. Em Python, muitas dessas estruturas já vêm prontas, o que facilita muito a vida do desenvolvedor. Ao longo do nosso conteúdo aqui, vamos abordar esses facilitadores. Que são listas, tuplas, dicionários e conjuntos. Além disso, vamos entender conceitos importantes como escopo de variáveis, uso de funções, estruturas como pilhas e filas. Tudo isso com exemplos claros e aplicáveis. Vamos começar pelas listas, que são provavelmente a estrutura mais usada em Python. Uma lista é uma coleção ordenada, mutável e capaz de armazenar vários valores juntos. Isso significa que os dados tem posição definida é que podemos adicionar, remover e alterar elementos com facilidade. Pense em uma lista de compras. Ela tem uma ordem, pode mudar ao longo do tempo e agrupa itens relacionados. Em Python, a ideia é exatamente a mesma, só que aplicada a dados. Nesta linha de código apresentada na tela, criamos uma lista chamada notas. Os colchetes indicam que temos uma lista e cada valor é separado por vírgula. Com isso, conseguimos armazenar várias informações relacionadas dentro de uma única estrutura. Listas permitem tratar conjuntos de dados como uma única entidade lógica, facilitando cálculos, validações e processamento em massa. Quando criamos uma lista, cada elemento recebe uma posição chamada de índice. Mas é importante lembrar, o primeiro índice é sempre zero e índices negativos acessam elementos a partir do final. São detalhes que costumam confundir iniciantes, mas com prática fica natural. Para acessar um elemento da lista, usamos o seu índice e para modificar, simplesmente atribuímos o novo valor naquela posição. Isso mostra porque dizemos que listas são mutáveis, já que o conceito pode mudar ao longo do programa. Python oferece métodos para realizar essas ações nos elementos das listas, como `append`, `remove`, `pop` e `insert`. Neste exemplo apresentado em tela, criamos uma lista vazia e depois usamos o método `append` para adicionar elementos à lista. Agora vamos às tuplas. À primeira vista, elas parecem listas, mas existe uma diferença fundamental: tuplas são imutáveis, ou seja, depois de criadas não podem ser alteradas. E você pode se perguntar: "por que a imutabilidade é importante?" Porque ela traz segurança. Ela garante que certos dados não sejam alterados acidentalmente durante a execução do programa, por isso, tuplas são muito usadas para representar informações fixas. Assim como as listas, as tuplas permitem acesso por índice, fatiamento e iteração. O que muda é que métodos de alteração não existem justamente para preservar os dados. Ao comparar listas e tuplas, vale uma regra simples: se os dados mudam, estamos falando de listas, se os dados não mudam, estamos abordando as tuplas. Essa decisão impacta diretamente na clareza e na segurança do código, conforme destaca o autor Luciano Ramalho, no livro *Python Fluent*. a correta utilização de estruturas imutáveis é determinante para garantir robustez e eficiência no código. Outra estrutura importante em Python são os dicionários. Eles organizam informações em pares de chave e valor. Isso permite acessar dados pelo significado e não pela posição. Vamos pensar juntos em estruturas de dicionários utilizados em situações do mundo real. um cadastro de aluno, por exemplo, tem nome, matrícula, idade. Ou seja, cada informação tem um rótulo, exatamente como funciona o dicionário em Python. Ao acessar um valor em um dicionário, usamos a chave correspondente. Isso torna o código mais legível e fácil de manter, especialmente em sistemas maiores. Em operações importantes com dicionários, Python permite verificar se uma chave existe, percorrer chaves e valores e usar métodos seguros como `get` e os dois parênteses. Agora, quando o assunto é modificação de dicionários, precisamos alterar um valor existente e adicionar uma nova informação ao dicionário. Isso mostra como essa estrutura é flexível. Considere, por exemplo, um sistema de controle de estoque em uma loja, você poderia armazenar produtos com seus preços e quantidades. Perceba que esse exemplo ilustra dicionários aninhados, onde um dicionário contém outros dicionários como valores. Essa flexibilidade permite representar estruturas complexas de dados de forma intuitiva. A tratabilidade dessas estruturas é fundamental para sistemas modernos. Ao discutirem a importância de estruturas de dados bem planejadas. sobre os conjuntos ou sets, são coleções que não aceitam valores repetidos. Eles são muito úteis quando queremos eliminar duplicatas, comparar grupos de dados e verificar pertencimento. Os conjuntos suportam operações matemáticas como união e interseção. Essas operações são rápidas e muito usadas em análise de dados e lógica de sistemas. Além das listas, tuplas e conjuntos, ainda existem outros conceitos em Python que são importantes de você conhecer, como por exemplo, o escopo. Ele define a região do código onde a variável pode ser acessada. Compreender escopo é fundamental para escrever código organizado, evitar conflitos de nomes e criar funções reutilizáveis. Python possui principalmente dois escopos: o global e o local. Variáveis globais são

acessíveis em todo o programa. Enquanto variáveis locais existem apenas dentro da função onde foram definidas. Evitar modificar variáveis globais de dentro de funções é uma boa prática que melhora a legibilidade e a manutenção do código. Um código bem estruturado em termos de escopo é mais seguro, previsível e colaborativo. Em Python, as funções criam escopos próprios que ajudam a organizar e estruturar programas encapsulando uma determinada lógica entre si. Podemos pensar nelas como caixas pretas que recebem entradas. Realizam processamento e retornam saídas sem afetar o estado global do programa. Outro importante conceito em Python é a pilha. Ela é uma estrutura de dados linear que segue o princípio LIFO, sigla para Last in first out, No que traduzimos para último a entrar a primeiro a sair. Imagine uma pilha de pratos na qual você sempre coloca pratos no topo e só remove do topo. Ou seja, o último prato a entrar na pilha será o primeiro a sair. Isso é o que ocorre com dados organizados em pilha no Python. As operações fundamentais de uma pilha são push, que adiciona um elemento no topo; pop, que facilita remover e retornar o elemento do topo; e peak, que significa visualizar o topo sem remover. Já a estrutura de dados do tipo fila ou queue é uma estrutura linear que segue o princípio FIFO, que é a sigla para a expressão em inglês First in First Out, ou seja, primeiro a entrar, primeiro a sair. Imagine, por exemplo, uma fila de banco. Nela, o primeiro cliente a chegar é o primeiro a ser atendido. E é assim que os dados estruturados em fila funcionam. As operações fundamentais com essas estruturas são enqueue, que adiciona ao final e dequeue, que remove do início. Estrutura de dados bem escolhidas são apenas metade da história. A outra metade é documentar e estruturar seu código para que seja compreensível para você mesmo e para outros desenvolvedores. Comentários significativos e nomes descritivos são essenciais. Ao documentar estruturas de dados, deixe claro qual é o propósito da estrutura. Que tipos de dados contém e como deve ser manipulada. Uma doc string bem escrita em uma função explica a seu propósito, parâmetros e retorno. Bom, até aqui nós vimos como organizar dados de forma eficiente usando Python. Esses conceitos são a base para algoritmos mais complexos, estruturas avançadas e aplicações profissionais. Quanto mais você pratica a estrutura de dados, mais natural a programação se torna. Aproveite e separe um tempinho para colocar em prática o que vimos. Até breve!